

Case Study #2 Report

Student names: *Augustus Kaufmann, Aaron Duda*

Course: *Machine Learning* – Professor: *Dr. Sanmukh Kuppannagari*

Due date: *May 6th, 2026*

Chosen Clustering Function and Parameters

To select the best model suitable for determining the proper clustering of maritime vessel signals, we first considered using some of the algorithms in the scikit-learn module on clustering found at <https://scikit-learn.org/stable/modules/clustering.html>. We immediately eliminated hierarchical clustering methods since we are not trying to determine a cluster hierarchy. Additionally we do not use DBSCAN, HDBSCAN, OPTICS, BIRCH, or gaussian mixtures for a few reasons: (1) we want to cluster all samples and not label some as outliers, and (2) these algorithms provide additional capabilities which are not necessary for grouping maritime vessel signals. This left us with K-means, affinity propagation, and mean shift. For the sake of simplicity and the fact that the latter two are not scalable with respect to the number of samples, we decided to use the K-means algorithm.

To find the optimal number of samples, we set the following parameters, varied as specified.

1. `n_clusters` (Varied)

This parameter represents the number of vessels in the data, and so this number along with the correct assignment of each sample to the correct vessel is the problem we are trying to solve. We do this by comparing the results of different values of this parameter and performing silhouette analysis to choose the optimal parameter.

2. `init`

This parameter controls how we initialize the centroids. There is either a random approach, or an optimized greedy approach called `k-means++`. We set this parameter to `k-means++` since this is the default and performs faster than the traditional randomized approach.

3. `n_init`

This parameter controls how many times to initialize and run the k-means algorithm, mitigating the risk of a bad initial assignment. We set this to 10 since we are optimizing for correct cluster assignment over speed of convergence.

4. `max_iter`

This parameter controls how many iterations of k-means are allowed until the algorithm forcibly halts. We leave this at the default value 300.

5. tol

This parameter is essentially the convergence threshold, and we leave this at the default value of 0.0001

6. algorithm

This parameter determines which variant of k-means to run, either the classical formulation called `lloyd` or a version more efficient in some cases called `elkan`. We decided to go with the default `lloyd` algorithm since we are willing to sacrifice some speed for greater accuracy and simplicity.

Solution Method

Feature Preprocessing

In this section, we briefly go through how we preprocessed the data, considering details such as dropped columns, extracted features, and our standardization method.

VID. To run k-means against the test data sets `set1` and `set2`, we drop the VID column so that it does not effect the clustering. After clustering we use the groudtruth cluster assignments dictated by VID to determine the accuracy of the model.

OBJECT_ID. We also drop the OBJECT_ID column, since it provides no relevant information to help with clustering vessel reports.

SEQUENCE_DTTM. For this feature, we break it down into 3 new features: `HOURS`, `MINUTES`, and `SECONDS`. This is a form understandable to our model as opposed to the UTC format.

Standardization. To ensure feature scale did not affect the outcome of the k-means clustering, we use the `StandardScaler` from `scikit-learn` to transform the data such that each feature has its mean set to 0 and variance to 1. This helps curb the effect of some features dominating the distortion calculation and also maintains the distribution shape.

Insights and Clarity